```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
import pandas as pd


digits = load_digits()
X, y = digits.data, digits.target


X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, shuffle=True, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)

print("Train:", X_train.shape, "Val:", X_val.shape, "Test:", X_test.shape)
```

```
Train: (1257, 64) Val: (270, 64) Test: (270, 64)
```

```
class MyKNN:
    def __init__(self, k=3, distance="euclidean"):
        self.k = k
        self.distance = distance

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def compute_distance(self, x1, x2):
        if self.distance == "euclidean":
            return np.sqrt(np.sum((x1 - x2) ** 2))
        elif self.distance == "manhattan":
            return np.sum(np.abs(x1 - x2))
        elif self.distance == "cosine":
            num = np.dot(x1, x2)
            denom = np.linalg.norm(x1) * np.linalg.norm(x2)
            return 1 - (num / denom)
        else:
            raise ValueError("Unknown distance metric!")

    def predict(self, X):
        preds = []
        for x in X:
            distances = [self.compute_distance(x, x_train) for x_train in self.X_train]
            k_indices = np.argsort(distances)[:self.k]
            k_labels = self.y_train[k_indices]
            preds.append(np.bincount(k_labels).argmax())
        return np.array(preds)



best_model = None
best_score = 0
best_params = {}

for dist in ["euclidean", "manhattan", "cosine"]:
    for k in [1, 3, 5, 7]:
        knn = MyKNN(k=k, distance=dist)
        knn.fit(X_train, y_train)
        y_pred_val = knn.predict(X_val)
        acc = accuracy_score(y_val, y_pred_val)
        if acc > best_score:
            best_score = acc
            best_model = knn
```

```
                best_params = {"k": k, "distance": dist}

print("Best KNN Params:", best_params, "Val Accuracy:", best_score)
```

```
Best KNN Params: {'k': 5, 'distance': 'cosine'} Val Accuracy: 0.9962962962962963
```

```
y_pred_test_knn = best_model.predict(X_test)


knn_results = {
    "Accuracy": accuracy_score(y_test, y_pred_test_knn),
    "F1": f1_score(y_test, y_pred_test_knn, average="macro"),
    "Precision": precision_score(y_test, y_pred_test_knn, average="macro"),
    "Recall": recall_score(y_test, y_pred_test_knn, average="macro"),
}
print("KNN Test Results:", knn_results)
```

```
KNN Test Results: {'Accuracy': 0.9851851851851852, 'F1': 0.9841747881147622, 'Precision': 0.9857950872656754, 'Recall
```

```
models = {
    "DecisionTree": DecisionTreeClassifier(max_depth=10),
    "LogisticRegression": LogisticRegression(max_iter=2000),
    "SVM": SVC(kernel="rbf", C=10, gamma=0.01)
}


results = {"KNN": knn_results}


for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred_test = model.predict(X_test)
    results[name] = {
        "Accuracy": accuracy_score(y_test, y_pred_test),
        "F1": f1_score(y_test, y_pred_test, average="macro"),
        "Precision": precision_score(y_test, y_pred_test, average="macro"),
        "Recall": recall_score(y_test, y_pred_test, average="macro"),
    }



df_results = pd.DataFrame(results).T
print(df_results)
```

```
                    Accuracy        F1  Precision    Recall
KNN                 0.985185  0.984175   0.985795  0.983667
DecisionTree        0.837037  0.831251   0.833524  0.834963
LogisticRegression  0.970370  0.969373   0.969909  0.969266
SVM                 0.759259  0.800435   0.927778  0.759435
```